Projeto CG

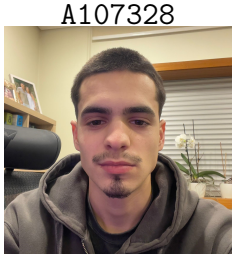
Grupo TP-06

Universidade do Minho
Licenciatura em Engenharia Informática
Computação Gráfica - 2025/26

Gonçalo Costa
A107381

Diogo Costa
A107328

Lourenço Martins
A106849



Gonçalo Sá
A107376

27 de fevereiro de 2026

Conteúdo

1 Introdução **2**

2 Estruturação do Projeto **2**

3 Gerador de Primitivas (Generator) **2**
3.1 Plano (Plane) 2
3.2 Caixa (Box) 2
3.3 Esfera (Sphere) 3
3.4 Cone 4

4 Motor de Renderização (Engine) **4**
4.1 Funcionalidades e Estrutura de Dados 5
4.2 Otimização de Memória 5

5 Compilação e Execução **5**

6 Resultados e Demonstração **5**
6.1 Visualização das Primitivas 6
6.2 Interação e Modos de Desenho 6
6.3 Análise de Desempenho 7

7 Conclusão **7**

1 Introdução

Este relatório detalha o desenvolvimento da primeira fase da unidade curricular de Computação Gráfica. O objetivo principal consistiu na criação de duas aplicações fundamentais:

- **Generator:** Responsável pelo cálculo sistemático de vértices para primitivas geométricas e exportação para ficheiros `.3d`.
- **Engine:** Uma aplicação de visualização capaz de interpretar ficheiros de configuração XML e renderizar os modelos previamente gerados recorrendo à API OpenGL.

2 Estruturação do Projeto

O projeto foi organizado de forma modular para facilitar a manutenção e escalabilidade:

- **engine**: Implementação do motor de renderização e lógica de visualização.
- **generator**: Algoritmos de geração de coordenadas para as primitivas.
- **tinyXML/**: Biblioteca externa utilizada para o *parsing* de ficheiros XML.
- **CMakeLists**: Ficheiro de configuração responsável por orquestrar a compilação de todo o projeto, garantindo a geração dos dois executáveis independentes (engine e generator) e a correta linkagem com as bibliotecas gráficas (OpenGL e GLUT).

3 Gerador de Primitivas (Generator)

A unidade *Generator* é uma ferramenta de linha de comandos desenvolvida em C++ cujo propósito é o cálculo geométrico e a geração de malhas de triângulos para primitivas gráficas. O output é um ficheiro de texto customizado contendo o número total de vértices e as respetivas coordenadas cartesianas (x, y, z).

3.1 Plano (Plane)

O plano é a primitiva mais simples, gerada no plano horizontal XZ. Para permitir uma tesselagem controlada, o plano é dividido em N divisões por eixo, resultando em N^2 quadrados, onde cada quadrado é composto por dois triângulos. Considerando um plano de lado L e div divisões, a largura de cada célula é $\Delta = \frac{L}{div}$. As coordenadas de um vértice (i, j) são dadas por:

$$\begin{cases} x = -\frac{L}{2} + i \cdot \Delta \\ y = 0 \\ z = -\frac{L}{2} + j \cdot \Delta \end{cases} \quad \text{onde } i, j \in \{0, \ldots, div\} \tag{1}$$

3.2 Caixa (Box)

A geração da caixa (cubo) expande o conceito do plano para três dimensões. Para cada uma das 6 faces, é gerada uma grelha de $div \times div$ triângulos. A lógica matemática

baseia-se em fixar uma coordenada (correspondente à face) e variar as outras duas. Por exemplo, para as faces perpendiculares ao eixo Y (topo e base), as equações são:

$$V_{topo} = \left(-\frac{L}{2} + i\Delta,\ \frac{L}{2},\ -\frac{L}{2} + j\Delta\right) \quad , \quad V_{base} = \left(-\frac{L}{2} + i\Delta,\ -\frac{L}{2},\ -\frac{L}{2} + j\Delta\right) \tag{2}$$

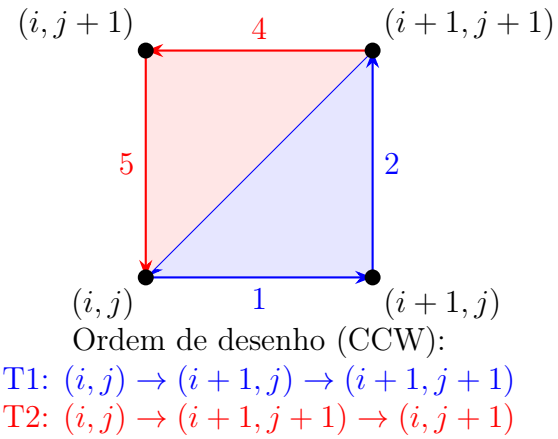


Figura 1: Esquema de triangulação da grelha. A ordem dos vértices (sentido anti-horário) define a normal da face para efeitos de *Backface Culling*.

Detalhe de Implementação: A implementação recorre a uma função auxiliar para desenhar cada face. Um aspeto crucial foi garantir que a ordem de declaração dos vértices respeita a regra da mão direita (*Counter-Clockwise*). No caso das faces "traseiras"ou "inferiores", a ordem dos vértices foi invertida programaticamente para assegurar o correto funcionamento do *Backface Culling* do OpenGL, garantindo que as normais apontam sempre para o exterior do cubo.

3.3 Esfera (Sphere)

A esfera é gerada utilizando coordenadas esféricas, convertidas posteriormente para coordenadas cartesianas. A superfície é dividida em fatias verticais (*slices*) e camadas horizontais (*stacks*). Para um raio r, com α (longitude) e β (latitude), as coordenadas de qualquer ponto são:

$$\begin{cases} x = r \cdot \cos(\beta) \cdot \sin(\alpha) \\ y = r \cdot \sin(\beta) \\ z = r \cdot \cos(\beta) \cdot \cos(\alpha) \end{cases} \tag{3}$$

Onde os ângulos variam segundo as divisões:

- $\alpha = i \cdot \frac{2\pi}{slices}, \quad i \in \{0, \ldots, slices\}$
- $\beta = -\frac{\pi}{2} + j \cdot \frac{\pi}{stacks}, \quad j \in \{0, \ldots, stacks\}$

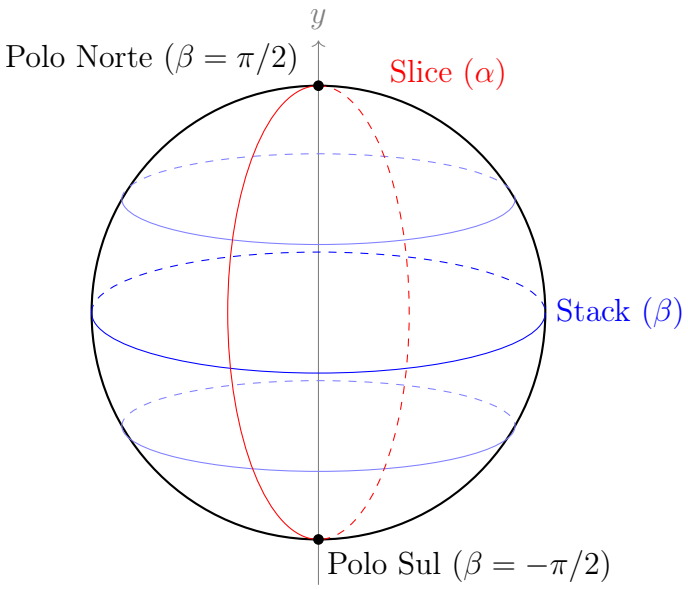


Figura 2: Representação das divisões da esfera. As linhas azuis representam as *stacks* (variação de β) e as vermelhas as *slices* (variação de α).

Detalhe de Implementação: O algoritmo trata os polos norte e sul como casos especiais. Nestas regiões, os quadriláteros da malha colapsam num único ponto. O código implementado evita explicitamente a criação de triângulos degenerados (com área zero) nestas extremidades, otimizando a geometria final.

3.4 Cone

O cone é definido pelo raio da base R, altura H, e o número de *slices* e *stacks*. A superfície lateral é calculada reduzindo o raio proporcionalmente à medida que subimos no eixo Y. O raio numa determinada camada j (*stack*) é dado por:

$$r_j = R \cdot \left(1 - \frac{j}{stacks}\right) \tag{4}$$

As coordenadas cartesianas para a lateral são então:

$$\begin{cases} x = r_j \cdot \sin(i \cdot \frac{2\pi}{slices}) \\ y = j \cdot \frac{H}{stacks} \\ z = r_j \cdot \cos(i \cdot \frac{2\pi}{slices}) \end{cases} \tag{5}$$

A base é fechada com um conjunto de triângulos ("fatias de tarte") que ligam o centro $(0, 0, 0)$ aos pontos do perímetro inferior, garantindo que o sólido é fechado. À semelhança da esfera, o algoritmo omite a geração do segundo triângulo na última stack do cone para evitar um triângulo degenerado no pico.

4 Motor de Renderização (Engine)

O *engine* foi desenvolvido utilizando a biblioteca **GLUT** para a gestão da janela e *callbacks*, e o **TinyXML-2** para o processamento robusto dos ficheiros de configuração.

4.1 Funcionalidades e Estrutura de Dados

A aplicação lê do ficheiro XML as definições da janela e da câmara (posição, *lookAt*, *up*, *fov*, *near*, *far*), bem como a lista de modelos a carregar.

- **Visualização:** Implementou-se o desenho de eixos coloridos (X-Vermelho, Y-Verde, Z-Azul) para facilitar a orientação espacial na cena.
- **Interação:** O sistema permite alternar o modo de rasterização em tempo real através das teclas:
  - **'f'**: Modo *Fill* (Sólido).
  - **'l'**: Modo *Line* (Wireframe).
  - **'p'**: Modo *Point* (Vértices).
  - **'w', 's'**: Alteram a escala dos modelos no eixo Y.
  - **'a', 'd'**: Aplicam rotação aos modelos em torno do eixo Y.
  - **Setas direcionais**: Permitem a navegação da câmara (aproximar, afastar e movimento lateral).

4.2 Otimização de Memória

Ao nível da implementação, os modelos são carregados para estruturas `std::vector` de pontos. Uma otimização importante implementada foi a utilização do método `reserve()` dos vetores. Ao ler o cabeçalho do ficheiro `.3d` (que contém o número total de vértices), a aplicação reserva antecipadamente a memória necessária. Esta abordagem evita múltiplas realocações dinâmicas de memória durante o processo de leitura, tornando o carregamento de modelos complexos significativamente mais eficiente.

5 Compilação e Execução

O projeto utiliza **CMake** para automatizar o processo de compilação, gerando dois executáveis distintos.

Listing 1: Exemplos de execução

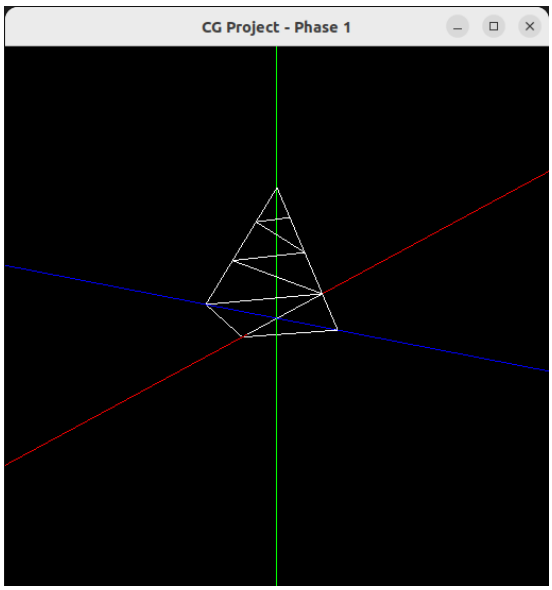
```
# Gerar uma esfera
./generator box 2 3 box_2_3.3d

# Executar o engine
./engine "../test files/test_files_phase_1/test_1_4.xml"
```

6 Resultados e Demonstração

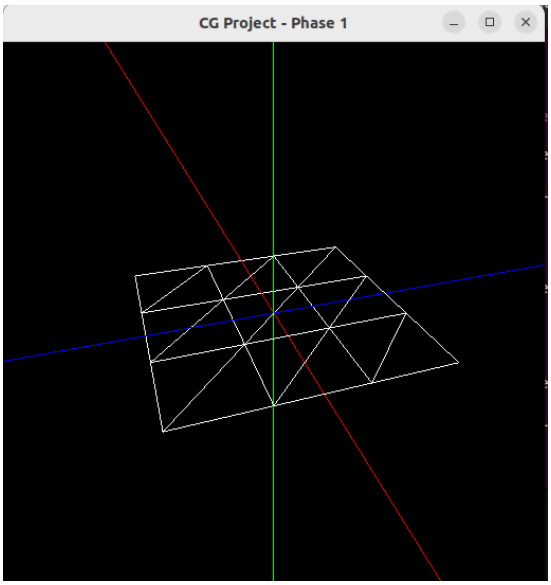
Nesta secção, apresentam-se os resultados visuais obtidos através da renderização das primitivas geradas pelo *Generator* e visualizadas no *Engine* desenvolvido. Os testes comprovam a correta implementação dos algoritmos de cálculo de vértices e a integração com o motor gráfico.

6.1 Visualização das Primitivas

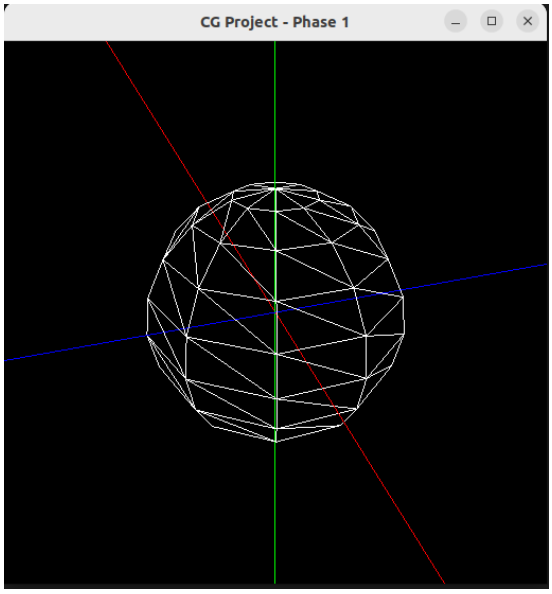
As figuras seguintes ilustram as quatro primitivas base solicitadas, renderizadas em modo *Line* (wireframe) para evidenciar a triangulação e a subdivisão das malhas (*slices* e *stacks*).



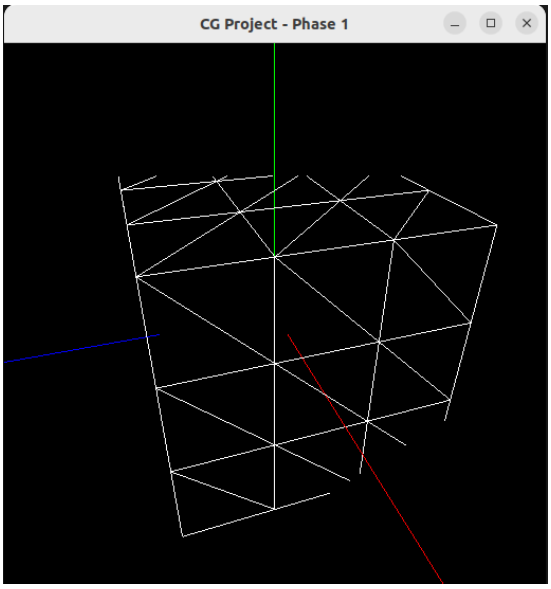
(a) Cone com base e lateral subdividida.



(b) Plano (XZ) com subdivisões.



(c) Esfera gerada por coordenadas esféricas.



(d) Caixa com 6 faces tesseladas.

Figura 3: Demonstração visual das primitivas geradas (Modo Wireframe).

6.2 Interação e Modos de Desenho

O motor permite a alternância dinâmica entre diferentes modos de visualização através do teclado (teclas 'f', 'l', 'p'), conforme demonstrado na Figura 4.

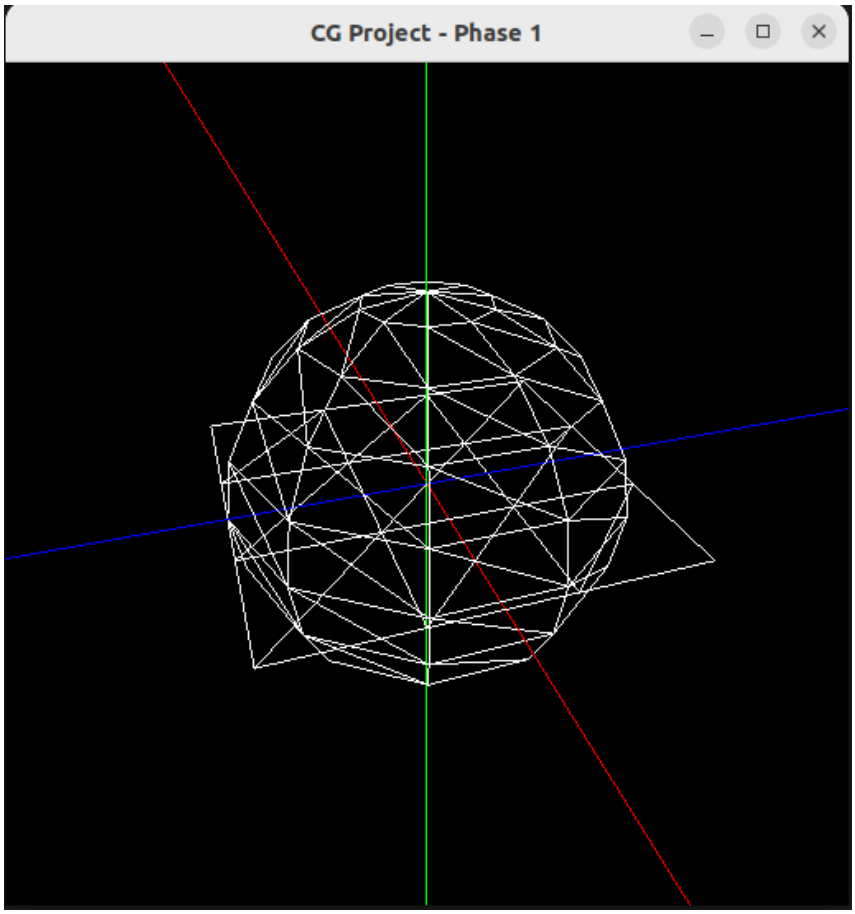


Figura 4: Exemplo de renderização em modo *Line* com eixos de referência coloridos.

6.3 Análise de Desempenho

Todos os modelos foram carregados com sucesso a partir dos ficheiros `.3d`. O *parsing* do ficheiro XML via *TinyXML-2* permitiu configurar a câmara e os modelos de forma automática, garantindo que a projeção e a perspetiva correspondem aos requisitos do enunciado.

7 Conclusão

Nesta fase inicial, foi possível estabelecer as bases do motor gráfico. A correta implementação da exportação de vértices e a integração com o TinyXML garantem a flexibilidade necessária para as fases seguintes, onde serão introduzidas transformações geométricas e estruturas de cena mais complexas.